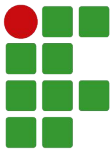


Flexbox em CSS

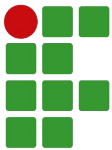
Prof. Christiano Lima Santos



Objetivos

Espera-se que cada aprendiz alcance os seguintes objetivos:

- ❑ Compreender os conceitos básicos envolvidos em layouts CSS utilizando Flexbox;
- ❑ Ser capaz de formatar o front-end de websites utilizando Flexbox;
- ❑ Compreender vantagens e desvantagens no uso do Flexbox e outras abordagens para layout responsivo utilizando CSS;
- ❑ Desenvolver um projeto web para compor seu portfólio profissional.

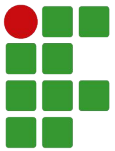


Conteúdo do curso

- ❑ Introdução ao Flexbox
- ❑ Conceitos Fundamentais
- ❑ Propriedades do Contêiner Flexível
- ❑ Propriedades dos Itens Flexíveis
- ❑ Exemplos Práticos de Layouts com Flexbox
- ❑ Flexbox vs Outras Técnicas de Layout
- ❑ Erros Comuns e Dicas de Flexbox

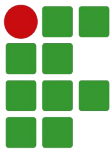
Introdução ao Flexbox

Parte 1



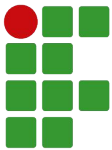
Sumário

- ❏ 1.1 O que é o Flexbox?
- ❏ 1.2 Vantagens do Flexbox
- ❏ 1.3 Quando usar o Flexbox?



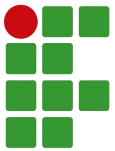
1.1 O que é o Flexbox?

- ❑ É um modelo de layout em CSS criado para facilitar a organização dos elementos em uma página, principalmente **alinhamento, espaçamento e distribuição de itens dentro de um contêiner**;
- ❑ Antes, criar layouts com colunas, centralizar verticalmente elementos ou distribuir itens igualmente era um processo trabalhoso, dependente de "gambiarras" com *float*, *position* e *margin*.



1.2 Vantagens do Flexbox

- ❑ **Alinhamento fácil:** tanto horizontal quanto vertical;
- ❑ **Distribuição automática de espaço** entre os elementos;
- ❑ **Reordenação visual** dos itens sem alterar o HTML;
- ❑ **Responsividade simplificada**, pois os itens se ajustam dinamicamente ao tamanho do contêiner;
- ❑ **Sintaxe clara e legível**, com propriedades específicas para contêiner (pai) e para itens (filhos).

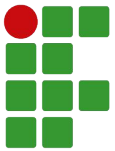


1.3 Quando usar o Flexbox?

- ❑ Use **Flexbox** quando o layout for **em uma direção (linha ou coluna)**;
- ❑ Use **CSS Grid** quando o layout precisar ser **em duas direções (linhas e colunas ao mesmo tempo)**.

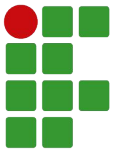
Conceitos Fundamentais

Parte 2



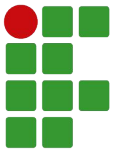
Sumário

- ❏ 2.1 Elemento Pai (Flex Container)
- ❏ 2.2 Elementos Filhos (Flex Items)
- ❏ 2.3 Eixos do Flexbox



2.1 Elemento Pai (Flex Container)

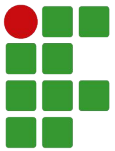
- ❑ O primeiro passo é definir um **elemento pai** como um contêiner flexível, aplicando a propriedade **display: flex** ou **display: inline-flex** em um elemento HTML;
- ❑ A partir desse momento, todos os **elementos filhos diretos** passam a ser tratados como **itens flexíveis**.



2.1 Elemento Pai (Flex Container)

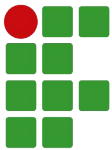
```
<style>
  .container {
    display: flex;
  }
</style>

<div class="container">
  <div>Item 1</div>
  <div>Item 2</div>
  <div>Item 3</div>
</div>
```



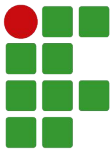
2.2 Elementos Filhos (Flex Items)

- ❑ Os elementos filhos diretos de um contêiner flexível são chamados de **Flex Items** e são afetados pelas propriedades do contêiner (como **justify-content** e **align-items**) e também podem ter propriedades próprias (como **flex-grow**, **order**, etc.).



2.2 Elementos Filhos (Flex Items)

- ❑ **Importante:** apenas os filhos diretos são considerados itens flexíveis. Elementos aninhados (dentro de outros filhos) **não são afetados** pelas regras do Flexbox, a menos que também sejam transformados em contêineres flexíveis.



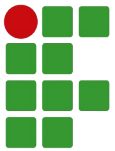
2.3 Eixos do Flexbox

❑ Main Axis (Eixo Principal)

- ❑ Eixo ao longo do qual os itens flexíveis são organizados, definido pela propriedade **flex-direction**. Por padrão, ele vai da **esquerda para a direita** (em linhas horizontais).

❑ Cross Axis (Eixo Cruzado)

- ❑ Eixo **perpendicular** ao eixo principal. Se o eixo principal é horizontal, o eixo cruzado é vertical (de cima para baixo), e vice-versa.

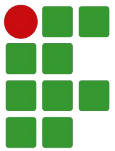


2.3 Eixos do Flexbox

```
.container-row {  
  display: flex;  
  flex-direction: row; /* Eixo principal: horizontal */  
}  
  
.container-column {  
  display: flex;  
  flex-direction: column; /* Eixo principal: vertical */  
}
```

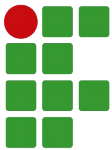
Propriedades do Contêiner Flexível

Parte 3



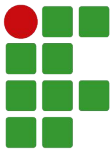
Sumário

- ❏ 3.1 `display: flex` e `display: inline-flex`
- ❏ 3.2 `flex-direction`
- ❏ 3.3 `flex-wrap`
- ❏ 3.4 `flex-flow`
- ❏ 3.5 `justify-content`
- ❏ 3.6 `align-items`
- ❏ 3.7 `align-content`



3.1 display: flex e display: inline-flex

- ❑ **display: flex** – o contêiner é exibido como um **bloco** (similar ao `display: block`), ocupando toda a largura disponível e começando em uma nova linha;
- ❑ **display: inline-flex** – o contêiner é tratado como um **elemento inline** (similar ao `display: inline-block`).



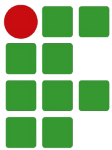
Exemplo com display: flex

— — —

```
<style>
.container-flex {
  display: flex;
  background-color: #eee;
}

.container-flex div {
  padding: 10px;
  background-color: lightblue;
  margin: 5px;
}
</style>

<div class="container-flex">
  <div>Item 1</div>
  <div>Item 2</div>
  <div>Item 3</div>
</div>
```



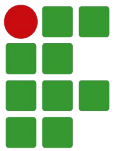
Exemplo com display: inline-flex

— — —

```
<style>
.container-inline-flex {
  display: inline-flex;
  background-color: #eee;
}

.container-inline-flex div {
  padding: 10px;
  background-color: lightgreen;
  margin: 5px;
}
</style>

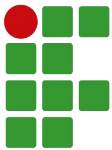
<div>Texto antes
  <span class="container-inline-flex">
    <div>Item A</div>
    <div>Item B</div>
  </span>
  Texto depois</div>
```



3.2 flex-direction

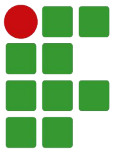
- ❑ Define a **direção do eixo principal**, ou seja, o modo como os itens serão organizados dentro do contêiner;

```
.container {  
  display: flex;  
  flex-direction: row; /* padrão */  
}
```



Valores possíveis para flex-direction

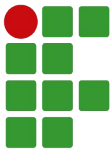
- ❑ **row** – itens organizados em **linha**, da esquerda para a direita (padrão);
- ❑ **row-reverse** – linha, mas da direita para a esquerda;
- ❑ **column** – itens organizados em **coluna**, de cima para baixo;
- ❑ **column-reverse** – coluna, mas de baixo para cima.



3.3 flex-wrap

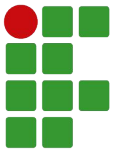
- ❑ Define se os itens devem **quebrar linha** quando não houver espaço suficiente;

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
}
```



Valores possíveis para flex-wrap

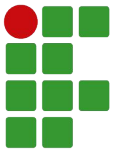
- ❑ **nowrap** – os itens **não quebram linha** (padrão);
- ❑ **wrap** – os itens quebram linha e seguem para baixo;
- ❑ **wrap-reverse** – os itens quebram linha, mas na direção oposta (de baixo para cima).



3.4 flex-flow

- ❑ É uma **forma abreviada** de declarar **flex-direction** e **flex-wrap** ao mesmo tempo;

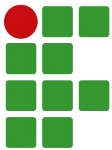
```
.container {  
  display: flex;  
  flex-flow: row wrap;  
}
```



3.5 justify-content

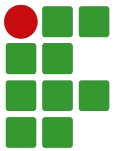
- ❑ Controla o **alinhamento dos itens no eixo principal** (horizontal por padrão). Define como o espaço extra é distribuído entre os itens;

```
.container {  
  display: flex;  
  justify-content: center;  
}
```



Valores possíveis para justify-content

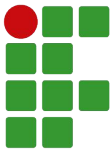
- ❑ **flex-start** - alinha os itens ao **início** do eixo (padrão);
- ❑ **flex-end** - alinha ao **final** do eixo;
- ❑ **center** - **centraliza** os itens;
- ❑ **space-between** - espaço igual **entre** os itens;
- ❑ **space-around** - espaço igual **ao redor** dos itens;
- ❑ **space-evenly** - espaço **exatamente igual** antes, entre e depois dos itens.



3.6 align-items

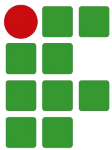
- ❑ Controla o **alinhamento dos itens no eixo cruzado** (vertical por padrão);

```
.container {  
  display: flex;  
  align-items: stretch; /* padrão */  
}
```



Valores possíveis para align-items

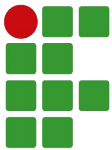
- ❑ **stretch** – os itens são esticados para ocupar toda a altura do contêiner (padrão);
- ❑ **flex-start** – alinha ao topo do contêiner;
- ❑ **flex-end** – alinha à base do contêiner;
- ❑ **center** – centraliza verticalmente;
- ❑ **baseline** – alinha os itens pela linha de base do texto.



3.7 align-content

- ❑ É usada **apenas quando há quebra de linha** (flex-wrap: wrap). Controla o **alinhamento das linhas** no eixo cruzado (vertical por padrão);

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
  align-content: center;  
}
```

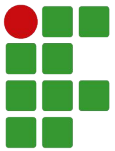


Valores possíveis para align-content

- ❑ **flex-start** - As linhas de itens são agrupadas no início do eixo cruzado;
- ❑ **flex-end** - As linhas de itens são agrupadas no final do eixo cruzado;
- ❑ **center** - As linhas de itens são centralizadas no eixo cruzado;
- ❑ **space-between** - O espaço disponível é distribuído igualmente entre as linhas;
- ❑ **space-around** - O espaço é distribuído ao redor de cada linha (meio espaço nas extremidades);
- ❑ **stretch** - As linhas de itens são esticadas para ocupar todo o espaço disponível no eixo cruzado.

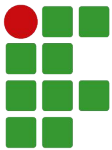
Propriedades dos Itens Flexíveis

Parte 4



Sumário

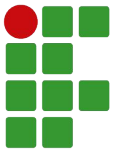
- ❏ 4.1 order
- ❏ 4.2 flex-grow
- ❏ 4.3 flex-shrink
- ❏ 4.4 flex-basis
- ❏ 4.5 flex
- ❏ 4.6 align-self



4.1 order

- ❑ Define a **ordem de exibição** dos itens no layout, **sem alterar a ordem no HTML**;
- ❑ Como funciona:
 - ❑ O valor padrão é 0;
 - ❑ Itens com menor valor de order aparecem antes;
 - ❑ Útil para **reorganizar visualmente** os elementos sem mudar a estrutura do código (por exemplo, no smartphone aparecerá em uma ordem, no computador em outra).

```
.item {  
    order: 1;  
}
```

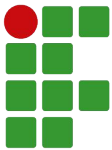


Exemplo com order

— — —

```
<style>
  .container-flex {
    display: flex;
  }
  .order-1 {
    order: 1;
  }
  .order-2 {
    order: 2;
  }
</style>

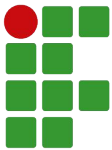
<div class="container-flex">
  <div class="order-2">Item A</div>
  <div class="order-1">Item B</div>
</div>
```



4.2 flex-grow

- ❑ Controla quanto um item **pode crescer** em relação aos outros, **quando há espaço disponível** no contêiner;
- ❑ Como funciona:
 - ❑ O valor padrão é 0, ou seja, o item **não cresce** além do tamanho necessário;
 - ❑ Se vários itens tiverem flex-grow definido, o espaço livre será dividido **proporcionalmente**.

```
.item {  
  flex-grow: 1;  
}
```

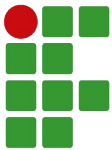


Exemplo com flex-grow

— — —

```
<style>
  .container-flex {
    display: flex;
    justify-content: stretch;
    align-items: center;
    background-color: #aaa;
  }
  .container-flex div {
    padding: 10px;
    background-color: lightgreen;
    margin: 5px;
  }
  .grow-2 { flex-grow: 2; }
  .grow-1 { flex-grow: 1; }
  .grow-0 { flex-grow: 0; }
</style>

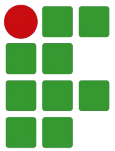
<div class="container-flex">
  <div class="grow-2">Item A</div>
  <div class="grow-1">Item B</div>
  <div class="grow-0">Item C</div>
</div>
```



4.3 flex-shrink

- ❑ Define quanto um item pode **encolher** quando não há espaço suficiente no contêiner;
- ❑ Como funciona:
 - ❑ O valor padrão é 1, o que significa que o item **pode encolher**;
 - ❑ Se definido como 0, o item **não encolherá**, mesmo se ultrapassar o limite do contêiner (*overflow*).

```
.item {  
  flex-shrink: 1;  
}
```

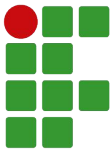


Exemplo com flex-shrink

— — —

```
<style>
  .container-flex {
    display: flex;
    justify-content: stretch;
    align-items: center;
    background-color: #aaa;
  }
  .container-flex div {
    padding: 10px;
    background-color: lightgreen;
    margin: 5px;
    width: 300px;
  }
  .shrink-2 { flex-shrink: 2; }
  .shrink-1 { flex-shrink: 1; }
  .shrink-0 { flex-shrink: 0; }
</style>

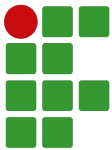
<div class="container-flex">
  <div class="shrink-2">Item A</div>
  <div class="shrink-1">Item B</div>
  <div class="shrink-0">Item C</div>
</div>
```



4.4 flex-basis

- ❑ Define o **tamanho inicial** do item antes que o espaço seja distribuído, substituindo temporariamente a largura (*width*) ou altura (*height*), dependendo da direção do Flexbox;
- ❑ Como funciona:
 - ❑ O valor pode ser em px, %, rem, etc.
 - ❑ O padrão é auto (usa *width* ou *height* do item, se definido).

```
.item {  
  flex-basis: 200px;  
}
```

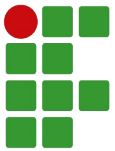


Exemplo com flex-basis

— — —

```
<style>
  .container-flex {
    display: flex;
    align-items: center;
    align-content: flex-start;
    background-color: #aaa;
  }
  .container-flex div {
    padding: 10px;
    background-color: lightgreen;
    margin: 5px;
    flex-basis: 200px;
  }
  .shrink-0 { flex-shrink: 0; }
</style>

<div class="container-flex">
  <div class="shrink-0">Item A</div>
  <div class="shrink-0">Item B</div>
  <div class="shrink-0">Item C</div>
</div>
```

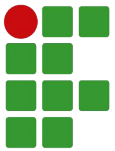


4.5 flex

— — —

- ❑ É uma **propriedade abreviada** para definir os valores de *flex-grow*, *flex-shrink* e *flex-basis*;

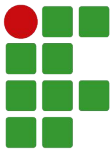
```
.item {  
  flex: 1 1 200px;  
}
```



4.6 align-self

- ❑ Permite **sobrescrever o alinhamento vertical** (**align-items**) apenas para um item específico, no eixo cruzado;

```
.item {  
  align-self: center;  
}
```

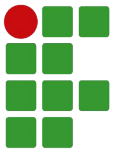


Valores possíveis para align-self

- ❑ **auto** - herda o valor de *align-items* (padrão);
- ❑ **flex-start** - alinha ao início do eixo cruzado;
- ❑ **flex-end** - alinha ao final do eixo cruzado;
- ❑ **center** - centraliza verticalmente;
- ❑ **stretch** - estica o item para ocupar todo o espaço disponível;
- ❑ **baseline** - alinha pela linha de base do texto.

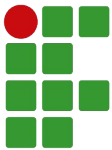
Exemplos Práticos de Layouts com Flexbox

Parte 5



Sumário

- ❏ 5.1 Navegação Horizontal (Menu)
- ❏ 5.2 Cartões em Grade
- ❏ 5.3 Cabeçalho com Logotipo e Menu
- ❏ 5.4 Página com Sidebar
- ❏ 5.5 Centralização de Formulários
- ❏ 5.6 Rodapé com Colunas

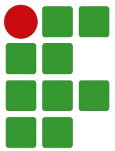


5.1 Layout de Navegação Horizontal (Menu)

— — —

```
<style>
  .menu {
    display: flex;
    justify-content: space-around;
    background-color: #444;
    padding: 10px;
  }
  .menu a {
    color: white;
    text-decoration: none;
    padding: 8px 16px;
  }
</style>

<nav class="menu">
  <a href="#">Início</a>
  <a href="#">Sobre</a>
  <a href="#">Serviços</a>
  <a href="#">Contato</a>
</nav>
```

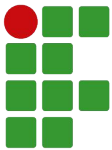


5.2 Layout de Cartões em Grade

— — —

```
<style>
  .cards {
    display: flex;
    gap: 16px;
  }
  .card {
    flex: 1;
    background-color: #e0e0e0;
    padding: 20px;
    text-align: center;
    border-radius: 8px;
  }
</style>

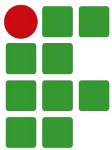
<div class="cards">
  <div class="card">Card 1</div>
  <div class="card">Card 2</div>
  <div class="card">Card 3</div>
</div>
```



5.3 Layout de Cabeçalho com Logotipo e Menu

```
<style>
  .topbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    background: #1a1a1a;
    padding: 10px 20px;
    color: white;
  }
  .topmenu {
    display: flex;
    gap: 16px;
  }
  .topmenu a {
    color: white;
    text-decoration: none;
  }
</style>

<header class="topbar">
  <div class="logo">LOGO</div>
  <nav class="topmenu">
    <a href="#">Home</a>
    <a href="#">Blog</a>
    <a href="#">Contato</a>
  </nav>
</header>
```

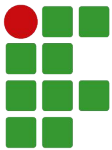


5.4 Layout de Página com Sidebar

— — —

```
<style>
  .layout {
    display: flex;
  }
  .sidebar {
    flex: 1;
    background: #f0f0f0;
    padding: 20px;
  }
  .content {
    flex: 3;
    background: #ffffff;
    padding: 20px;
  }
</style>

<div class="layout">
  <aside class="sidebar">Barra Lateral</aside>
  <main class="content">Conteúdo Principal</main>
</div>
```

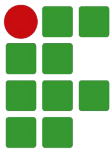


5.5 Layout com Centralização de Formulários

— — —

```
<style>
  html, body {
    margin: 0;
  }
  .container {
    display: flex;
    justify-content: center; /* Centraliza horizontalmente */
    align-items: center;     /* Centraliza verticalmente */
    height: 100vh;          /* Ocupa 100% da altura da tela */
    background-color: #f2f2f2;
  }
  form.login {
    background-color: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
  }
</style>

<body>
  <div class="container">
    <form class="login">
      <h2>Login</h2>
      <p><input type="text" placeholder="Usuário" required></p>
      <p><input type="password" placeholder="Senha" required></p>
      <p><button type="submit">Entrar</button></p>
    </form>
  </div>
</body>
```



5.6 Layout de Rodapé com Colunas

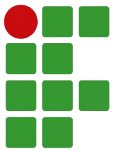
— — —

```
<style>
  .footer {
    display: flex;
    justify-content: space-between;
    background: #222;
    color: white;
    padding: 20px;
  }
</style>

<footer class="footer">
  <div>Links Úteis</div>
  <div>Redes Sociais</div>
  <div>Contato</div>
</footer>
```

Flexbox vs Outras Técnicas de Layout

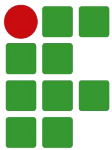
Parte 6



Sumário

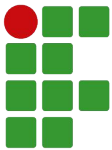
- ❏ 6.1 Flexbox vs Float/Clearfix

- ❏ 6.2 Flexbox vs Grid



6.1 Flexbox vs Float/Clearfix

- ❑ Antes do Flexbox, o posicionamento de elementos lado a lado em páginas web frequentemente utilizava a propriedade **float**;
- ❑ Originalmente criado para fazer com que imagens “flutuassem” dentro de blocos de texto, passou a ser usado como solução de layout – o que gerou diversas limitações e a necessidade de “gambiarrras” como o uso de **clearfix** para corrigir comportamentos inesperados.

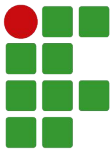


Exemplo de Layout com float e clearfix

— — —

```
<style>
  .container {
    width: 100%;
    overflow: auto; /* clearfix tradicional */
    background-color: #f0f0f0;
  }
  .container::after { /* clearfix moderno, mais seguro */
    clear: both;
    content: "";
    display: table;
  }
  .box {
    float: left;
    width: 30%;
    margin: 1%;
    padding: 20px;
    background-color: #4CAF50;
    color: white;
  }
</style>

<div class="container">
  <div class="box">Caixa 1</div>
  <div class="box">Caixa 2</div>
  <div class="box">Caixa 3</div>
</div>
```

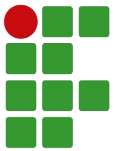


Exemplo de Layout equivalente com Flexbox

— — —

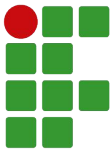
```
<style>
.container {
  display: flex;
  justify-content: space-between;
  background-color: #f0f0f0;
}
.box {
  width: 30%;
  padding: 20px;
  background-color: #2196F3;
  color: white;
}
</style>

<div class="container">
  <div class="box">Caixa 1</div>
  <div class="box">Caixa 2</div>
  <div class="box">Caixa 3</div>
</div>
```



6.2 Flexbox vs Grid

- ❑ **Flexbox** é ideal para layouts **unidimensionais** (em linha ou coluna). Distribui os elementos com base na direção definida e oferece controle detalhado sobre o espaçamento, o alinhamento e o tamanho relativo dos itens;
- ❑ **CSS Grid** é voltado para layouts **bidimensionais**, ou seja, que exigem controle simultâneo em linhas e colunas. Permite construir grades complexas, sobrepor elementos e criar áreas nomeadas para posicionar conteúdos com maior precisão.

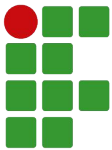


Exemplo de Layout com Flexbox

— — —

```
<style>
.flex-container {
  display: flex;
  justify-content: space-between;
  background-color: #e0e0e0;
}
.flex-item {
  width: 30%;
  padding: 20px;
  background-color: #009688;
  color: white;
}
</style>

<div class="flex-container">
  <div class="flex-item">Item 1</div>
  <div class="flex-item">Item 2</div>
  <div class="flex-item">Item 3</div>
</div>
```



Exemplo de Layout equivalente com Grid

— — —

```
<style>
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
  background-color: #e0e0e0;
  padding: 10px;
}
.grid-item {
  background-color: #3F51B5;
  color: white;
  padding: 20px;
}
</style>
```

```
<div class="grid-container">
  <div class="grid-item">Item 1</div>
  <div class="grid-item">Item 2</div>
  <div class="grid-item">Item 3</div>
</div>
```